Pre Study

Literature review

Bruno Carchia
carchia@kth.se

May 5, 2026

Introduction

This section provides a critical analysis of the literature and technical reports that informed the selection of the methodologies and technologies employed in this research. Each reviewed paper contributed specifically to the architectural design or the strategic formulation of the Reinforcement Learning agent. To ensure clarity, each paper is presented using the following structure:

- **Core Summary:** A brief overview of the research objectives and findings of the paper.
- **Technical Contribution:** An explanation of how the paper's insights directly influenced the development of this thesis.

1 Fundamentals of Reinforcement Learning

Reinforcement Learning (RL) is a distinct machine learning paradigm characterized by several foundational principles:

1. **Evaluative Feedback:** Unlike supervised learning, there is no external supervisor providing labeled data. The system relies entirely on a reward signal, which indicates the quality of an action (good or bad) without providing the absolute optimal target.
2. **Delayed Consequences:** Feedback is rarely instantaneous; actions taken at the current time step may have profound effects on rewards received many steps in the future.
3. **Sequential Decision-Making:** Time is a critical variable. RL deals with sequential states and actions where data points are highly correlated, fundamentally breaking the independent and identically distributed (i.i.d.) assumption common in classical machine learning.

4. **Agent-Environment Interaction:** An active agent continuously moves through and interacts with a dynamic environment.

5. **Active Sampling:** The agent's actions directly influence the subsequent data it observes, creating a continuous feedback loop between behavior and data generation.

A standard reinforcement learning system is defined by the following core components:

1. **State (S_t):** A mathematical function of the history H_t (a sequence of sequential observations, actions, and rewards, defined as $H_t = O_1, R_1, A_1, \ldots, A_{t-1}, O_t, R_t$) used to determine subsequent dynamics. States can be categorized into three types:
   a) *Environment State (S_t^e):* The internal representation used by the environment to dictate state transitions and emissions.
   b) *Agent State (S_t^a):* The internal representation utilized by the agent to summarize historical data and inform future action selection.
   c) *Information/Markov State (S_t^M):* A formal mathematical property indicating that the state sufficiently retains all necessary historical information. A state is Markovian [1] if the future is conditionally independent of the past, given the present: $\mathbb{P}[S_{t+1} \mid S_t] = \mathbb{P}[S_{t+1} \mid S_1, \ldots, S_t]$.

2. **Action (A_t):** The behavioral output executed by the agent.

3. **Reward (R_t):** A scalar feedback signal indicating the immediate utility of the agent's behavior at time step t. The primary objective of the agent is to maximize the cumulative sum of these expected rewards over time.

4. **Policy (π):** The agent's behavioral mapping from states to actions, formulated as $\pi(a|s) = \mathbb{P}[A_t = a \mid S_t = s]$. In a stochastic environment, this defines a probability distribution over the action space.

5. **Value Function ($v(s)$):** A prediction of future cumulative rewards, estimating the long-term desirability of being in a particular state or taking a specific action: $v(s) = \mathbb{E}[G_t \mid S_t = s]$, where G_t represents the expected return.

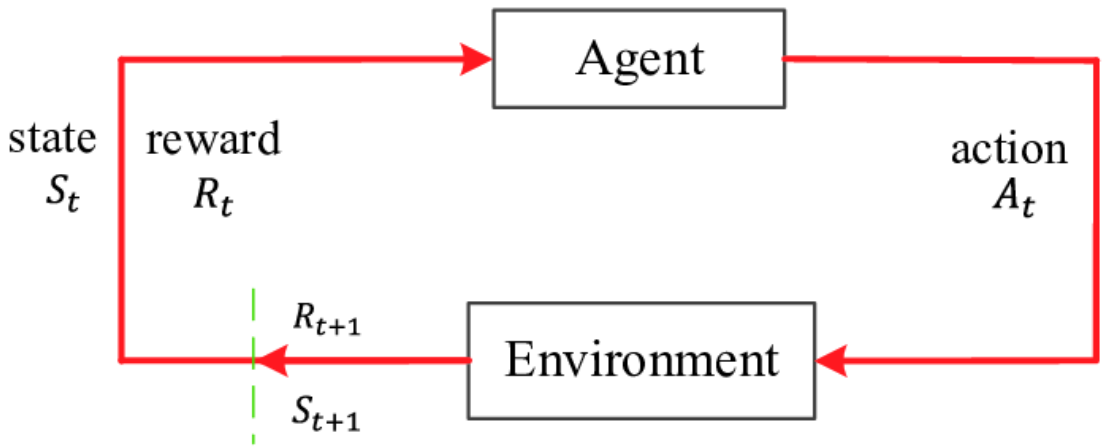


Figure 1: Agent-Environment Interaction in a Reinforcement Learning System. Taken from Jiang et al., 2019 [2].

The nature of the environment dictates the agent's observation capabilities. An environment is *fully observable* if the agent can directly observe the complete environment state ($S_t^a = S_t^e$), formally modeled as a Markov Decision Process (MDP). Conversely, if the agent only has access to a limited view of the state ($S_t^a \neq S_t^e$), the system is modeled as a Partially Observable Markov Decision Process (POMDP).

Furthermore, RL agents are fundamentally categorized by their algorithmic approach:

1. **Value-Based:** The agent approximates the value function, while the policy is implicitly derived (e.g., via an ϵ-greedy strategy).

2. **Policy-Based:** The agent explicitly parameterizes and optimizes the policy π without maintaining a separate value function.

3. **Actor-Critic:** A hybrid architecture utilizing both functions: the *actor* updates the policy, while the *critic* evaluates the policy by estimating the value function.

1.1 Markov Decision Process (MDP)

A Markov process constitutes a memoryless stochastic sequence of random states $S_1, S_2, \ldots$ that strictly adheres to the Markov Property [1]. An MDP extends this by introducing actions and rewards, formalized as the tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$:

- $\mathcal{S}$: A finite set of states.

- $\mathcal{A}$: A finite set of actions.

- $\mathcal{P}$: The state transition probability matrix, where $\mathcal{P}_{ss'}^a = \mathbb{P}[S_{t+1} = s' \mid S_t = s, A_t = a]$.

- $\mathcal{R}$: The expected reward function, defined as $\mathcal{R}_s^a = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$.
- $\gamma \in [0, 1]$: The discount factor, quantifying the present value of future rewards.

The fundamental goal in an MDP is to maximize the expected return G_t, defined as the total discounted cumulative reward:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \tag{1}$$

1.2 Bellman Expectation Equations

Value functions can be systematically decomposed into two constituent parts: the immediate expected reward and the discounted expected value of the successor state.

1. **State-Value Function ($v_\pi(s)$):** Evaluates the expected return starting in state s and strictly following policy π.

$$\begin{aligned} v_\pi(s) &= \mathbb{E}_\pi[G_t \mid S_t = s] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \mid S_t = s] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \dots) \mid S_t = s] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s] \end{aligned} \tag{2}$$

   By marginalizing over discrete actions, this function connects to the action-value function: $v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) q_\pi(s, a)$.

2. **Action-Value Function ($q_\pi(s, a)$):** Evaluates the expected return of taking action a in state s, and subsequently following policy π.

$$q_\pi(s, a) = \mathbb{E}_\pi[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a] \tag{3}$$

   Similarly, this can be expressed by averaging over the environmental dynamics: $q_\pi(s, a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_\pi(s')$.

By substituting these relationships, we derive the recursive **Bellman Expectation Equations**:

$$v_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a \mid s) \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_\pi(s') \right) \tag{4}$$

$$q_\pi(s, a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \sum_{a' \in \mathcal{A}} \pi(a' \mid s') q_\pi(s', a') \tag{5}$$

1.3 Bellman Optimality Equations

The primary objective of RL is to identify the optimal policy π_*, defined as the policy that yields the maximum possible state-value function $v_{\pi_*}(s) = \max_\pi v_\pi(s)$ and action-value function $q_{\pi_*}(s,a) = \max_\pi q_\pi(s,a)$ across all states. According to established theory [3], there always exists at least one optimal deterministic policy $\pi_* \geq \pi$, $\forall \pi$.

This principle leads to the **Bellman Optimality Equations**:

- For the optimal state-value function $v_*(s)$: The value of a state under the optimal policy must equal the expected return for the best available action. It requires a one-step look-ahead, maximizing over the action space and averaging over the stochastic transition dynamics determined by the environment.

$$v_*(s) = \max_a \left[\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_*(s') \right] \tag{6}$$

- For the optimal action-value function $q_*(s,a)$: The value of taking an action relies on the immediate reward plus the discounted expected value of the best possible action in the subsequent state.

$$q_*(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \max_{a'} q_*(s',a') \tag{7}$$

Due to the inherent non-linearity of the max operator, Bellman optimality equations generally lack a closed-form solution. Consequently, they necessitate iterative algorithmic solvers such as Value Iteration, Policy Iteration, Q-Learning, and SARSA.

1.4 Policy Gradient Methods

In contrast to value-based methods, which derive behavior implicitly from value estimates, policy gradient methods directly parameterize and optimize the policy itself. By representing the policy as a parameterized probability distribution, $\pi_\theta(a|s) = \mathbb{P}[a \mid s; \theta]$, the agent selects actions conditioned on both the current state and the parameter vector θ.

This direct parameterization offers several fundamental **advantages**:

- *Enhanced Convergence Properties:* The policy update follows the gradient smoothly, reducing the oscillatory behavior often seen in value-based approximations.
- *Continuous and High-Dimensional Action Spaces:* It naturally accommodates continuous action spaces without the computational bottleneck of calculating a global maximum over actions (as required by the max operator in Q-learning).
- *Stochastic Policies:* It allows the agent to learn and maintain stochastic policies, which is essential in environments with imperfect information or non-stationary dynamics.

Conversely, policy gradient approaches possess notable **disadvantages**:

- *Local Optima:* Gradient ascent guarantees convergence to a local optimum, which may not correspond to the global optimum.
- *High Variance:* Evaluating a policy natively via trajectory sampling is typically highly inefficient and suffers from significant variance, which can destabilize learning.

To optimize the parameters θ, an objective function $J(\theta)$ must be defined to quantitatively evaluate the quality of the policy. The formulation of $J(\theta)$ depends fundamentally on the nature of the environment:

- **Start Value (Episodic Environments):** For environments with a defined starting state s_1, the objective is the expected long-term return from that initial state:

$$J_1(\theta) = V^{\pi_\theta}(s_1) = \mathbb{E}_{\pi_\theta}[v_1] \tag{8}$$

  This metric asks: "If the episode begins at s_1 and strictly follows π_θ, what is the total expected reward?"

- **Average Value (Continuing Environments):** In non-episodic environments lacking a distinct start state, the objective evaluates the expected value of all states, weighted by the stationary distribution $d^{\pi_\theta}(s)$ of the Markov chain induced by π_θ:

$$J_{avV}(\theta) = \sum_s d^{\pi_\theta}(s) V^{\pi_\theta}(s) \tag{9}$$

- **Average Reward per Time-Step (Continuing Environments):** Alternatively, the objective can be framed as the expected immediate reward per time-step under the stationary distribution:

$$J_{avR}(\theta) = \sum_s d^{\pi_\theta}(s) \sum_a \pi_\theta(a|s) \mathcal{R}_s^a \tag{10}$$

  Here, $d^{\pi_\theta}(s)$ acts as a generalized probability distribution over the state space as time approaches infinity.

Remarkably, the gradient of the objective function takes the same underlying form regardless of which specific $J(\theta)$ is utilized.

$$
\begin{aligned}
\nabla_\theta J(\theta) &= \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(s,a)\ v_t\right] && \text{REINFORCE} \\
&= \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(s,a)\ Q^w(s,a)\right] && \text{Q Actor-Critic} \\
&= \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(s,a)\ A^w(s,a)\right] && \text{Advantage Actor-Critic} \\
&= \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(s,a)\ \delta\right] && \text{TD Actor-Critic} \\
&= \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(s,a)\ \delta e\right] && \text{TD}(\lambda)\text{ Actor-Critic} \\
G_\theta^{-1}\nabla_\theta J(\theta) &= w && \text{Natural Actor-Critic}
\end{aligned}
$$

Figure 2: The foundational architecture of Policy Gradient Algorithms, illustrating equivalent mathematical forms across different objective functions.

Policy Gradient Algorithms

The core mechanism of these algorithms relies on the Policy Gradient Theorem, which states that if the policy π_θ is differentiable, the parameters can be updated via stochastic gradient ascent. The general form of the gradient is:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a|s) Q^{\pi_\theta}(s,a)] \tag{11}$$

- **Monte Carlo Policy Gradient (REINFORCE):** To approximate the expected return without requiring a complete model of the environment dynamics, the REINFORCE algorithm relies on empirical returns v_t sampled from full trajectories as an unbiased estimate of the true action-value function $Q^{\pi_\theta}(s_t, a_t)$.

  $$\Delta\theta_t = \alpha \nabla_\theta \log \pi_\theta(a_t|s_t) v_t \tag{12}$$

  The gradient term $\nabla_\theta \log \pi_\theta(a_t|s_t)$, known as the score function, effectively steers the parameter updates to increase the probability of actions associated with higher empirical returns. However, the reliance on full Monte Carlo rollouts renders this approach computationally slow and introduces substantial variance, potentially destabilizing the learning process [3].

- **Actor-Critic Policy Gradient:** To alleviate the high variance inherent to Monte Carlo sampling, Actor-Critic architectures introduce a parameterized value function, termed the *Critic* ($Q_w(s,a) \approx Q^{\pi_\theta}(s,a)$), to explicitly estimate the expected return. This substitution yields the approximate policy gradient:

  $$\nabla_\theta J(\theta) \approx \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a|s) Q_w(s,a)] \tag{13}$$

  This architecture fundamentally comprises two interacting components:

  - The **Critic**, which updates its parameters w to accurately evaluate current state-action pairs by minimizing the temporal-difference error.

- The **Actor**, which updates the policy parameters θ in the direction of the gradient formulated by the Critic's evaluations.

Action-Value Actor-Critic with Baseline: In a standard formulation utilizing linear value function approximation ($Q_w(s,a) = \phi(s,a)^T w$), the iterative updates are defined as follows:

$$\delta = r + \gamma Q_w(s', a') - Q_w(s, a) \qquad \text{(TD Error)} \tag{14}$$

$$\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi_\theta(a|s) Q_w(s, a) \qquad \text{(Actor Update)} \tag{15}$$

$$w \leftarrow w + \beta \delta \phi(s, a) \qquad \text{(Critic Update)} \tag{16}$$

To further reduce the variance of the gradient estimator without introducing bias (i.e., without altering the expectation of the gradient), an arbitrary state-dependent baseline function $B(s)$ can be subtracted from the value estimate. This subtraction preserves the expected gradient because the expectation of the baseline term rigorously evaluates to zero:

$$\begin{aligned} \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a \mid s) B(s)] &= \sum_{s \in \mathcal{S}} d^{\pi_\theta}(s) \sum_{a \in \mathcal{A}} \nabla_\theta \pi_\theta(a \mid s) B(s) \\ &= \sum_{s \in \mathcal{S}} d^{\pi_\theta}(s) B(s) \nabla_\theta \left(\sum_{a \in \mathcal{A}} \pi_\theta(a \mid s) \right) \\ &= 0 \end{aligned} \tag{17}$$

The final equality holds because the sum over all action probabilities in a given state must equal 1 ($\sum_{a \in \mathcal{A}} \pi_\theta(a \mid s) = 1$), and the gradient of a constant is mathematically zero.

The theoretically optimal choice for this baseline is the state-value function $V^{\pi_\theta}(s)$, which quantifies the average expected performance starting from state s. Subtracting this baseline from the action-value function yields the **Advantage Function** $A^{\pi_\theta}(s, a)$:

$$A^{\pi_\theta}(s, a) = Q^{\pi_\theta}(s, a) - V^{\pi_\theta}(s) \tag{18}$$

By substituting the Advantage function into the objective, the modified policy gradient becomes:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a|s) A^{\pi_\theta}(s, a)] \tag{19}$$

This formulation profoundly stabilizes the learning trajectory. Instead of updating parameters based on the absolute magnitude of the expected return (Q), the gradient is scaled by the relative advantage (A). That is, how much better the chosen action performed compared to the average expected baseline for that state. This centering mechanism mitigates noise and yields significantly more efficient convergence.

1.5 Advantage Actor-Critic (A2C)

The Advantage Actor-Critic (A2C) algorithm is a synchronous, deterministic variant of the Asynchronous Advantage Actor-Critic (A3C) method. It operates by coordinating parallel environment data collection and executing batch updates. By simulating N parallel environments simultaneously and updating the policy based on the aggregated batch, A2C stabilizes the training process; rather than updating parameters based on a single, high-variance trajectory, the gradient is averaged across N diverse trajectories.

A fundamental distinction between A2C and traditional Q-Learning Actor-Critic architectures is that A2C does not directly approximate the action-value function $Q(s, a)$. Instead, it relies on empirical returns collected from the environments to compute the advantage function. This reliance on grounded trajectory data , rather than initially biased action-value approximator ,contributes significantly to the improved stability of A2C [4].

Algorithm Overview:

1. Initialize network parameters θ with random weights.
2. Execute N steps across parallel environments using the current policy π_θ, storing the transition tuples (s_t, a_t, r_t).
3. Initialize the return G_t: set $G_t = 0$ if a terminal state is reached; otherwise, bootstrap the value estimate $G_t = V_\theta(s_t)$.
4. For $i = t - 1$ down to t_{start} (iterating backward for computational efficiency):
   a) Accumulate the discounted return: $G_t \leftarrow r_i + \gamma G_t$.
   b) Compute the Advantage function, defined as the difference between the empirical return and the baseline expected return estimated by the critic: $A(s_i, a_i) = G_t - V_\theta(s_i)$.
   c) Accumulate the policy gradients (Actor update):
   $$\Delta\theta_\pi = \Delta\theta_\pi + \nabla_{\theta_\pi} \log \pi_{\theta_\pi}(a_i \mid s_i)\,(G_t - V_\theta(s_i))$$
   d) Accumulate the value gradients (Critic update) by minimizing the Mean Squared Error (MSE):
   $$\Delta\theta_v = \Delta\theta_v + \nabla_{\theta_v} \left(G_t - V_{\theta_v}(s_i)\right)^2$$
5. Update the neural network parameters in the direction of the accumulated gradients using an optimization algorithm (e.g., Adam).
6. Repeat from step 2 until convergence.

In practice, the A2C loss function is often augmented with an entropy regularization term. This term quantifies the uncertainty of the agent's policy, penalizing premature convergence to deterministic behavior. Higher entropy promotes exploration by encouraging the agent to sample diverse actions, whereas lower entropy signifies exploitation of known optimal actions.

1.6 Trust Region Policy Optimization (TRPO)

Schulman et al. [5] introduced Trust Region Policy Optimization (TRPO), an algorithm designed to ensure monotonic policy improvement. The problem is formalized as a Markov Decision Process (MDP) defined by the tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, r, \rho_0, \gamma \rangle$, where $\mathcal{S}$ is a finite state space, $\mathcal{A}$ is a finite action space, $\mathcal{P}$ represents the transition probability dynamics, r is the reward function, ρ_0 is the initial state distribution, and $\gamma \in (0, 1)$ is the discount factor.

For a stochastic policy $\pi : \mathcal{S} \times \mathcal{A} \to [0, 1]$, the expected discounted return $\eta(\pi)$ is defined as:

$$\eta(\pi) = \mathbb{E}_{s_0, a_0, \ldots} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t) \right] \tag{20}$$

where $s_0 \sim \rho_0(s_0)$, $a_t \sim \pi(a_t \mid s_t)$, and $s_{t+1} \sim \mathcal{P}(s_{t+1} \mid s_t, a_t)$.

The algorithm utilizes the standard definitions of the action-value function $Q_\pi(s_t, a_t)$, state-value function $V_\pi(s_t)$, and advantage function $A_\pi(s, a)$:

$$\begin{aligned} Q_\pi(s_t, a_t) &= \mathbb{E}_{s_{t+1}, a_{t+1}, \ldots} \left[\sum_{l=0}^{\infty} \gamma^l r(s_{t+l}) \right] \\ V_\pi(s_t) &= \mathbb{E}_{a_t, s_{t+1}, \ldots} \left[\sum_{l=0}^{\infty} \gamma^l r(s_{t+l}) \right] \\ A_\pi(s, a) &= Q_\pi(s, a) - V_\pi(s) \end{aligned} \tag{21}$$

Given a current policy π and a candidate policy $\tilde{\pi}$, the expected return of $\tilde{\pi}$ can be expressed in terms of the advantage over π (as proven in Lemma 1 of [5]):

$$\eta(\tilde{\pi}) = \eta(\pi) + \mathbb{E}_{s_0, a_0, \cdots \sim \tilde{\pi}} \left[\sum_{t=0}^{\infty} \gamma^t A_\pi(s_t, a_t) \right] \tag{22}$$

This formulation allows the evaluation of a new policy based on the baseline performance of the current policy. Introducing the unnormalized discounted visitation distribution $\rho_\pi(s) = \sum_{t=0}^{\infty} \gamma^t \mathbb{P}(s_t = s)$, which quantifies the visitation frequency of state s, the expected return can be rewritten as:

$$\eta(\tilde{\pi}) = \eta(\pi) + \sum_{s \in \mathcal{S}} \rho_{\tilde{\pi}}(s) \sum_{a \in \mathcal{A}} \tilde{\pi}(a \mid s) A_\pi(s, a) \tag{23}$$

Standard policy gradient methods suffer from two primary limitations: first, the gradient estimate is only a first-order local approximation; second, standard step sizes can lead to overshooting (resulting in performance collapse) or excessively slow convergence.

To address this, TRPO enforces a trust region to mathematically guarantee monotonic improvement, ensuring $\eta(\pi_{\theta_{k+1}}) \geq \eta(\pi_{\theta_k})$. The algorithm achieves this by maximizing a surrogate objective function $\mathcal{L}(\theta_k, \theta)$ subject to a strict constraint on the Kullback-Leibler (KL) divergence between the old and new policies:

$$\begin{aligned} \theta_{k+1} = \arg\max_{\theta} \; & \mathcal{L}(\theta_k, \theta) \\ \text{s.t. } & \bar{D}_{KL}(\theta \parallel \theta_k) \leq \delta \end{aligned} \tag{24}$$

Where the surrogate objective is defined as:

$$\mathcal{L}(\theta_k, \theta) = \mathbb{E}_{s,a\sim\pi_{\theta_k}} \left[\frac{\pi_\theta(a \mid s)}{\pi_{\theta_k}(a \mid s)} A^{\pi_{\theta_k}}(s,a) \right] \tag{25}$$

And the average KL-divergence penalty prevents destructively large updates:

$$\bar{D}_{KL}(\theta \parallel \theta_k) = \mathbb{E}_{s\sim\pi_{\theta_k}} \left[D_{KL} \left(\pi_\theta(\cdot \mid s) \parallel \pi_{\theta_k}(\cdot \mid s) \right) \right] \tag{26}$$

Because solving this constrained optimization problem exactly is intractable for large neural networks, TRPO employs a Taylor expansion to form a linear approximation of the objective and a quadratic approximation of the constraint:

$$\begin{aligned} \theta_{k+1} \approx \arg\max_{\theta} \; & g^T(\theta - \theta_k) \\ \text{s.t. } & \frac{1}{2}(\theta - \theta_k)^T H (\theta - \theta_k) \leq \delta \end{aligned} \tag{27}$$

Here, $g = \nabla_\theta \mathcal{L}(\theta_k, \theta)|_{\theta=\theta_k}$ is the gradient vector, and H is the Fisher Information Matrix (the Hessian of the average KL-divergence). Using Lagrangian duality [6, Ch. 2], the theoretical update step is derived as:

$$\theta_{k+1} = \theta_k + \alpha^j \sqrt{\frac{2\delta}{g^T H^{-1} g}} H^{-1} g \tag{28}$$

where $\alpha \in (0, 1)$ is a backtracking line-search coefficient. Since computing the inverse Hessian H^{-1} is computationally prohibitive for deep networks, TRPO uses the conjugate gradient algorithm [7] to efficiently estimate the natural gradient direction $x = H^{-1}g$ via matrix-vector products ($Hx = g$) without explicitly forming H.

Algorithm 1 Trust Region Policy Optimization

Input: initial policy parameters θ_0, initial value function parameters ϕ_0
Hyperparameters: KL-divergence limit δ, backtracking coefficient α, maximum number of backtracking steps K
for $k = 0, 1, 2, ...$ **do**
  Collect set of trajectories $\mathcal{D}_k = \{\tau_i\}$ by running policy $\pi_k = \pi(\theta_k)$ in the environment.
  Compute rewards-to-go $\hat{R}_t$.
  Compute advantage estimates, $\hat{A}_t$ (using any method of advantage estimation) based on the current value function V_{ϕ_k}.
  Estimate policy gradient as

$$\hat{g}_k = \frac{1}{|\mathcal{D}_k|} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t)|_{\theta_k} \hat{A}_t.$$

  Use the conjugate gradient algorithm to compute

$$\hat{x}_k \approx \hat{H}_k^{-1} \hat{g}_k,$$

  where $\hat{H}_k$ is the Hessian of the sample average KL-divergence.
  Update the policy by backtracking line search with

$$\theta_{k+1} = \theta_k + \alpha^j \sqrt{\frac{2\delta}{\hat{x}_k^T \hat{H}_k \hat{x}_k}} \hat{x}_k,$$

  where $j \in \{0, 1, 2, ...K\}$ is the smallest value which improves the sample loss and satisfies the sample KL-divergence constraint.
  Fit value function by regression on mean-squared error:

$$\phi_{k+1} = \arg\min_\phi \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \left(V_\phi(s_t) - \hat{R}_t \right)^2,$$

  typically via some gradient descent algorithm.
end for

Figure 3: TRPO algorithm pseudocode. Adapted from OpenAI Spinning Up [8].

1.7 Proximal Policy Optimization (PPO)

To overcome the computational complexity of second-order optimization in TRPO, Schulman et al. [9] introduced Proximal Policy Optimization (PPO). PPO retains the reliability and monotonic improvement guarantees of TRPO while significantly simplifying the implementation by utilizing purely first-order optimization techniques.

While the original paper proposes two variants (PPO-Penalty and PPO-Clip), the most widely adopted formulation is PPO-Clip. Rather than imposing a hard KL-divergence constraint, PPO-Clip alters the objective function to mathematically penalize

excessive divergence between the updated policy and the old policy.

Let $r_t(\theta)$ denote the probability ratio between the target and behavior policies:

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)} \tag{29}$$

The clipped surrogate objective function is formulated as:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta)\hat{A}_t,\, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right] \tag{30}$$

By explicitly clipping the probability ratio within the interval $[1-\epsilon, 1+\epsilon]$, the algorithm removes the incentive to move r_t outside of this bound when the advantage is positive, thereby establishing a local "trust region."

To compute the advantage estimates $\hat{A}_t$, PPO commonly integrates Generalized Advantage Estimation (GAE). GAE computes an exponentially weighted average of multi-step temporal-difference errors ($\delta_t = r_t + \gamma V_w(s_{t+1}) - V_w(s_t)$) to strike an optimal balance between bias and variance in the value estimations.

Algorithm 1 PPO-Clip

1: Input: initial policy parameters θ_0, initial value function parameters ϕ_0
2: **for** $k = 0, 1, 2, \ldots$ **do**
3: Collect set of trajectories $\mathcal{D}_k = \{\tau_i\}$ by running policy $\pi_k = \pi(\theta_k)$ in the environment.
4: Compute rewards-to-go $\hat{R}_t$.
5: Compute advantage estimates, $\hat{A}_t$ (using any method of advantage estimation) based on the current value function V_{ϕ_k}.
6: Update the policy by maximizing the PPO-Clip objective:

$$\theta_{k+1} = \arg\max_\theta \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \min \left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)} A^{\pi_{\theta_k}}(s_t, a_t),\quad g(\epsilon, A^{\pi_{\theta_k}}(s_t, a_t)) \right),$$

typically via stochastic gradient ascent with Adam.
7: Fit value function by regression on mean-squared error:

$$\phi_{k+1} = \arg\min_\phi \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \left(V_\phi(s_t) - \hat{R}_t \right)^2,$$

typically via some gradient descent algorithm.
8: **end for**

Figure 4: PPO algorithm pseudocode. Adapted from OpenAI Spinning Up [8].

Advantages of PPO:

- *Computational Efficiency:* By relying exclusively on first-order derivatives (gradients), PPO avoids the expensive Fisher Information Matrix computations of TRPO, leading to vastly superior wall-clock training times.

- *Stability and Robustness:* The surrogate clipping mechanism safely restricts step sizes, preventing the catastrophic performance collapse common in standard policy gradients.
- *Versatility:* It is highly scalable and consistently performs well across continuous and discrete action spaces.

Limitations of PPO:

- *Sample Inefficiency:* As an on-policy algorithm, PPO requires fresh data generated by the current policy for every update phase, rendering it inherently less sample-efficient than off-policy algorithms (e.g., SAC, DQN) which utilize experience replay buffers.
- *Hyperparameter Sensitivity:* Performance is sensitive to specific tuning parameters, particularly the clipping ratio (ϵ), learning rate, and GAE coefficient (λ).
- *Local Optima Vulnerability:* Because the clipping function only enforces a local trust region, noisy gradient estimates or excessively small batch sizes can still cause the policy to drift into suboptimal local minima.

2 An advanced learning environment and a scalable deep reinforcement learning approach for rolling stock circulation on urban rail transit line

This study [10] addresses the Rolling Stock Circulation (RSC) problem by formulating it as a multi-objective optimization task within a Deep Reinforcement Learning framework. The primary objectives identified are the minimization of fleet utilization, the balancing of maintenance workloads, and the stabilization of depot stock levels.

State Space Representation

The authors define a hierarchical state representation where the global state s^t is a concatenation of individual rolling stock statuses s_k^t. The local state for each asset k is defined as a six-dimensional feature vector:

$$s_k^t = \left(n_k^t, \bar{c}_k^t, \bar{b}_k^t, f_k^t, o_k^t, g_k^t\right)$$

This vector captures critical operational constraints, including cumulative trip counts (n_k^t), remaining service life ($\bar{c}_k^t$), and turnaround readiness (f_k^t). Notably, the inclusion of operation direction (o_k^t) and a timetable pointer (g_k^t) ensures the agent maintains the Markov property by encoding the physical feasibility of future assignments.

Action Space

The action space $A = \{1, 2, \ldots, |K|\}$ is discrete, where each action corresponds to the selection of a specific physical asset from the fleet. To handle the "hard constraints" of rail operations (e.g., assigning a busy train), the authors utilize a penalty , based

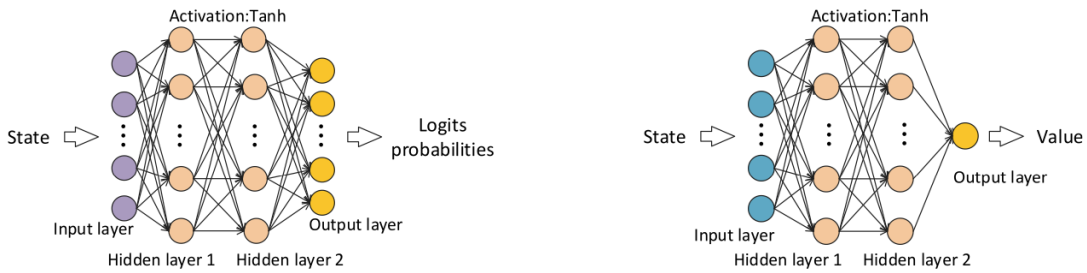
approach rather than strict action masking. This forces the agent to learn the underlying feasibility logic through the negative reinforcement of invalid selections.

Reward Mechanism

A dual-reward structure is employed to handle the credit assignment problem:

1. **Dense Step-wise Reward (r'):** Focused on immediate utilization efficiency and constraint satisfaction. It penalizes unnecessary depot extractions (r_a) and rewards the selection of idle assets with high turnaround times (r_v), promoting a "First-In, First-Out" logic.

2. **Sparse Terminal Reward (Z):** A weighted sum of normalized objectives calculated at the end of the operation horizon. This term targets fleet-wide fairness (workload standard deviation z_3) and depot equilibrium (z_2).



(a) Actor Network

(b) Critic Network

Figure 5: Architecture of the proposed Actor-Critic Neural Network. Taken from (Yang et al., 2025 [10])

Neural Network Architecture

The problem is solved using the Proximal Policy Optimization (PPO) algorithm. As illustrated in Figure 5, the Actor and Critic networks utilize a fully-connected Multi-Layer Perceptron (MLP) structure with hyperbolic tangent (tanh) activation functions.

Technical Contribution and Relevance to Thesis

This paper provides the foundational logic for my research, particularly in the structuring of sparse and dense rewards to balance local efficiency with global maintenance goals. However, while the authors utilize a global MLP that scales linearly with fleet size, my work improves upon this by implementing a **Shared-Weight MLP**. This ensures that the model is permutation-invariant and can generalize to varying fleet sizes without re-training—a significant limitation observed in the original paper's architecture.

Feature	Reference Paper	This Thesis
Algorithm	PPO (Actor-Critic)	PPO (Actor-Critic)
Action Space	Discrete (Fixed $\|K\|$)	Continuous Weights (Scalable)
NN Architecture	Global Flat MLP	Shared-Weight Architecture
Observation Space	Fixed Global Vector	Masked Relational Features
Maintenance Logic	Reactive (Fixed Thresholds)	Proactive (Weibull-based)
Simulation Type	Step-based	Event-driven

Table 1: Comparative analysis between the reference study and the proposed DRL approach.

3 Deep Reinforcement Learning for Integrated Train Scheduling and Rolling Stock Circulation

A significant contribution to integrated rail optimization is presented by Zhao et al. [11], who propose a Deep Reinforcement Learning (DRL) approach to solve the joint problem of train scheduling and rolling stock circulation. The authors utilize a Hybrid Proximal Policy Optimization (HPPO) algorithm to manage a hybrid action space characterized by both continuous scheduling decisions and discrete rolling stock assignments. The primary objective is to minimize a multi-objective cost function comprising passenger waiting time and operational expenses under complex constraints.

In this framework, the DRL agent acts as a dispatcher, sequentially determining section running times, station dwell times, and the allocation of rolling stock units—either by deploying units from the depot or reassigning units arriving from previous services.

State Space Representation

The state space S_n at step n captures the temporal and logistical dynamics of the network:

$$S_n = (d_{n,i}, a_{n,i}, l_{n,i}, o_{n,k}) \tag{31}$$

where $d_{n,i}$ and $a_{n,i}$ denote departure and arrival times at station i, $l_{n,i}$ represents the passenger queue length post-departure, and $o_{n,k}$ indicates the availability time of rolling stock unit k.

Action Space Representation

The action space A_n is hybrid, formulated as:

$$A_n = (h_{n+1}, r_{n+1,i}, e_{n+1,i}, f_{n+1,k}) \tag{32}$$

Here, the headway (h_{n+1}), section running times ($r_{n+1,i}$), and dwell times ($e_{n+1,i}$) are continuous variables, while the rolling stock assignment ($f_{n+1,k} \in \{0, 1\}$) is a discrete binary decision.

Reward Mechanism

The reward function is defined as the negative weighted sum of three components:

1. *Passenger Waiting Costs*: Calculated as the sum of waiting time for newly arrived passengers and those stranded due to capacity limits from previous services.

2. *Operational Costs*: Defined by the total cycle time for service n, calculated as $(a_{n,2I} - d_{n,I})$.

3. *Usage Costs*: Proportional to the total number of active rolling stock units $\sum_{k \in K} \eta_k$.

Neural Network Architecture

To handle the hybrid nature of the actions, the architecture employs a dual-actor system 6. A discrete actor (π_{θ_d}) outputs the probability distribution for rolling stock assignments, while a continuous actor (π_{θ_c}) parameters a multivariate Gaussian distribution to sample scheduling variables.

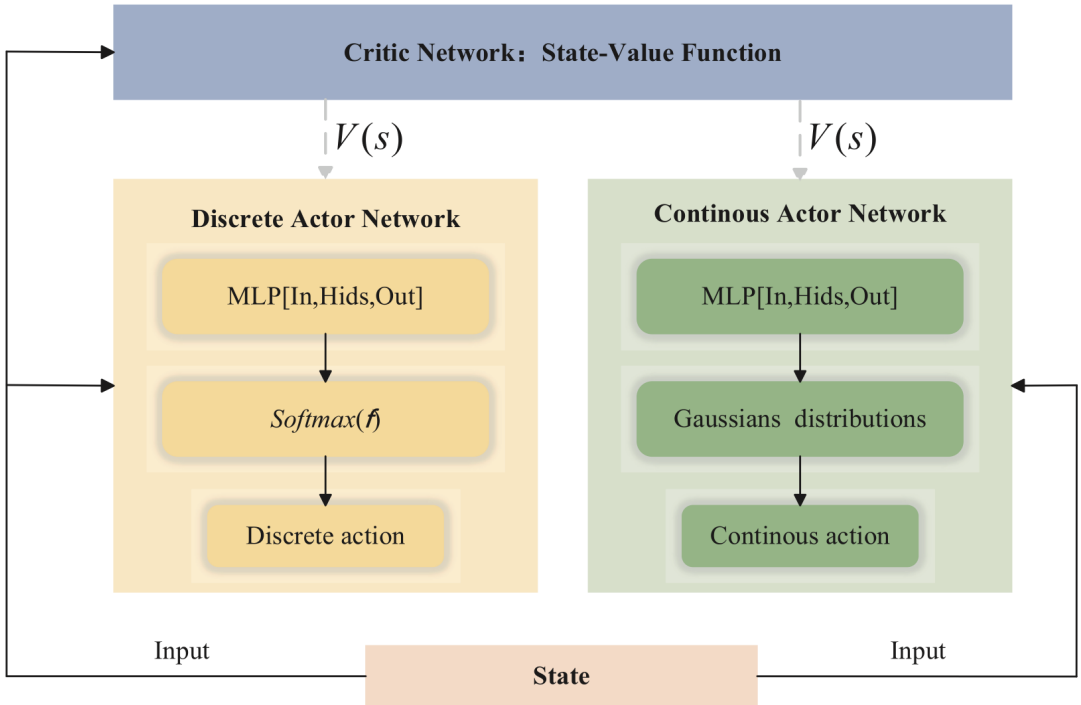


Figure 6: Architecture of the proposed Actor-Critic Neural Network. Taken from (Zhao et al., 2026 [11])

Relevance and Comparative Analysis

While the study by Zhao et al [11]. addresses a more geographically complex passenger network, it provides a vital methodological benchmark for this thesis. Both approaches utilize action masking to enforce operational constraints; however, several key distinctions exist. While Zhao et al. [11] optimize for passenger flows using a hybrid HPPO actor-critic model, this thesis focuses on freight logistics (specifically the transport of timber—within a two-line system).

Furthermore, instead of the dual-actor hybrid approach (HPPO) used in the cited study, this research employs a standard PPO algorithm with a single continuous actor. In this thesis, discrete requirements are handled by mapping continuous outputs to discrete values via a specialized mapping strategy, rather than maintaining a separate discrete actor network.

4 Solving high-speed rolling stock planning with an improved LSTM-PPO reinforcement learning algorithm

The work presented by Su et al. [12] investigates the Rolling Stock Planning (RSP) problem utilizing a PPO framework integrated with a Long Short-Term Memory (LSTM) network. To prevent the agent from converging on local optima, the authors incorporate a simulated annealing strategy, effectively balancing global exploration with local refinement. The choice of an LSTM architecture is specifically intended to capture temporal dependencies and provide the agent with a recursive memory of the episode's history.

A fundamental assumption of this study is the prioritization of a fixed maintenance threshold over train departures, ensuring safety constraints are satisfied. The operational environment is modeled with three stations and a fixed scheduling regime, establishing a scenario closely aligned with the parameters of this thesis.

State Space Representation

The state at time step t is defined by the history of assignments V_t and the current task x_t. The history V_t represents a comprehensive record of all train services already allocated to the v rolling stock units (RSUs), while x_t denotes the specific service currently awaiting assignment:

$$V_t = \left\{U_t^1, U_t^2, \ldots, U_t^v\right\} = \left\{\left\{u_1^1, u_2^1, \ldots, u_r^1\right\}, \ldots, \left\{u_1^v, u_2^v, \ldots, u_r^v\right\}\right\}$$

Action Space Representation

The agent is tasked with mapping the pending train service (x_t) to an available RSU (v_i). The resulting action is formulated as:

$$a_t = (v_i, x_t)$$

Consistent with the methodology adopted in this research, the action space is constrained by a filtering mechanism that automatically excludes actions violating operational requirements, such as maintenance schedules.

Reward Mechanism

The reward function is a multi-objective scalar derived from three primary components, each modulated by specific hyperparameters:

1. r_t^{connect}: A positive reward for establishing a feasible connection between successive train services on the same RSU.

2. r_t^{save}: An incentive aimed at minimizing the total number of RSUs required for operations.

3. r_t^{penalty}: A penalty term for the violation of hard constraints, primarily regarding fixed maintenance intervals.

Technical Contribution and Relevance to Thesis

The problem addressed by Su et al. [12] serves as a foundational benchmark for this thesis, although it represents a slightly simplified version of the real-world system as it focuses exclusively on RSU efficiency without considering passenger demand.

There is a significant alignment between this thesis and Su et al. [12] regarding the prioritization of system safety; both models enforce maintenance requirements as a higher priority than schedule adherence. However, the maintenance modeling differs in flexibility: while Su et al. [12] restricts maintenance to nighttime windows, this thesis allows the agent to execute early maintenance at any time to accommodate operational needs. Furthermore, while the present work omits dwell times as a deliberate simplification, this feature remains a viable candidate for future integration. Finally, although this thesis utilizes a Multi-Layer Perceptron (MLP) architecture rather than the LSTM employed by Su et al. [12], the algorithmic structure of their work provided significant inspiration for the development of my proposed solution.

Algorithm 1 LSTM-PPO for rolling stock planning.

Require: Initialized actor π_θ, critic V_ϕ, LSTM state h_0, learning rates α_θ, α_ϕ, clipping parameter ϵ, epochs K

Initialize rolling stock planning environment
for each training iteration **do**
  Collect episodes:
  for each circulation planning episode **do**
    Reset environment and initialize LSTM hidden state h_0
    for each train service step t **do**
      Observe current state s_t (e.g. RSU status, depot availability, task x_t)
      Compute action distribution $\pi_\theta(a_t \mid s_t, h_{t-1})$
      Sample assignment decision a_t
      Execute a_t, observe reward r_t and next state s_{t+1}
      Store (s_t, a_t, r_t, h_{t-1}) in buffer
      Update hidden state: $h_t \leftarrow \text{LSTM}(s_t, h_{t-1})$
    end for
  end for
  Estimate advantages $\hat{A}_t$ and returns $\hat{R}_t$ via GAE
  for K epochs **do**
    Sample mini-batches from buffer
    for each mini-batch **do**
      Compute probability ratio:

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t, h_{t-1})}{\pi_{\theta_{\text{old}}}(a_t \mid s_t, h_{t-1})}$$

      Compute clipped surrogate loss:

$$L^{\text{PPO}}(\theta) = \mathbb{E}_t\left[\min\left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right]$$

      Update actor parameters: $\theta \leftarrow \theta + \alpha_\theta \nabla_\theta L^{\text{PPO}}$
      Update critic loss: $L_V = \|V_\phi(s_t, h_{t-1}) - \hat{R}_t\|^2$
      Update critic parameters: $\phi \leftarrow \phi - \alpha_\phi \nabla_\phi L_V$
    end for
  end for
end for

Figure 7: PPO-LSTM Algorithm. Taken from (SU et al., 2026 [12])

5 Rolling Stock Planning with Integrated Maintenance Constraints: A Rolling Horizon MILP Approach

The work by Bröchin et al. [13] presents a robust optimization framework for rolling stock circulation that explicitly integrates multi-tier maintenance requirements. Unlike traditional "maintenance-first" approaches, which often decouple service scheduling from technical service, this study utilizes a Mixed-Integer Linear Programming (MILP) model within a rolling horizon framework to optimize short-term operational efficiency alongside long-term fleet reliability.

Operational Complexity and Horizon Segmentation

The model accounts for high-fidelity operational constraints, including specific maintenance categories (T001, ESA, and DLR) and workshop availability fluctuations due to holidays or capacity limits. A central feature of the methodology is the temporal segmentation of the decision-making window into two distinct phases: the Activity Horizon and the Maintenance Horizon, as illustrated in Figure 8.

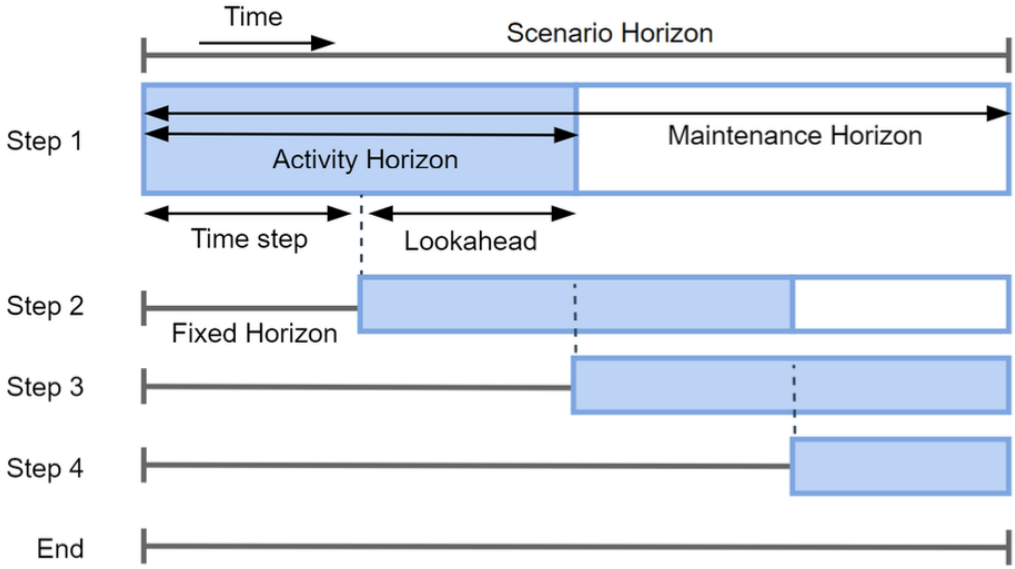


Figure 8: Temporal decomposition of the rolling horizon approach, distinguishing between detailed activity scheduling and expected maintenance requirements. Taken from Bröchin et al. (2023) [13].

By solving the MILP model at each step of the rolling horizon, the system performs a trade-off analysis between immediate revenue-generating activities and the expected future costs of maintenance neglect or suboptimal scheduling.

Multi-Layered Graph Architecture

The formulation utilizes a "layered flow" network architecture, which is critical for maintaining computational tractability. The graph is decomposed into:

- **Activity Flow Layer:** A directed graph $G_A = (V_A, A_A)$ where nodes represent service trips or maintenance slots, and arcs represent feasible transitions (turnarounds, repositioning, and coupling).
- **Maintenance Flow Layers:** Specialized layers for each maintenance type $t \in T$ that track the elapsed time/distance since the last service event, ensuring safety limits are not exceeded.

Objective Function

The optimization goal is to minimize a composite cost function representing both operational and technical expenditures:

$$\min \sum_{a \in A_A} c_a x_a + \sum_{t \in T} \sum_{a \in A_t} d_{a,t} w_{a,t} \tag{33}$$

This function penalizes deadheading (empty runs) and minimizes deviations from the original plan, effectively balancing economic performance with schedule stability.

Technical Contribution and Relevance to Thesis

The research by Bröchin et al. [13] is highly relevant as it demonstrates the necessity of integrating maintenance into the primary circulation plan to avoid "Maintenance Spikes." However, a significant limitation of their MILP-based approach is its computational latency, which makes it less suitable for the real-time, event-driven rescheduling required in dynamic freight environments. The papers is trying to resolve the following problem,: the assignment and reassignment of wagons to transport freight from origins to destinations within a railway network where there multiple stations and types of wagons. In contrast to their layered network flow, this thesis proposes an Actor-Critic DRL architecture. While Bröchin et al. [13] manage complexity through "commoditization" of flow, my approach utilizes a Shared-Weight MLP to achieve the same fleet-wide scalability without the need for solving expensive linear programs at every decision point. This allows for near-instantaneous rescheduling when stochastic events (such as the Weibull-distributed failures modeled in my work) occur.

6 Dynamic wagon assignment and reassignment for pick-up and delivery requests: A deep reinforcement learning framework to address uncertainty in locomotive availability

The research conducted by Habibi et al. (2026) [14] addresses the complexities of railway logistics through the application of Deep Reinforcement Learning (DRL), specifically utilizing the Proximal Policy Optimization (PPO) algorithm. The primary objective of their framework is the maximization of operational efficiency by ensuring the fulfillment of delivery requests while simultaneously minimizing logistical costs and temporal delays.

State Space Representation

The agent observes a discrete environment where the state s_t at any given time t is defined as:

$$s_t = (Bv^K_{t,m}, ya_{t,w,i})$$

This representation captures two critical components:

- The current inventory levels of freight across all pending requests.
- The distribution and availability of specific wagon types across the network's stations.

This state space design mirrors the requirements of complex railway environments, providing the necessary visibility into resource allocation and demand fulfillment.

Action Space Representation

A recurring challenge in railway scheduling—as observed in the model developed for this thesis—is the lack of scalability when the number of stations or assets increases. To mitigate the "curse of dimensionality" inherent in large-scale discrete action spaces, Habibi et al. introduce a set of twelve heuristic rules 11. These rules act as an abstraction layer for the agent, constraining the action space to high-level wagon assignment and re-assignment decisions. This approach contrasts with the raw weight-based selection mechanism utilized in my model, where the agent directly outputs weights for locomotive selection. While my model currently grants the agent greater freedom, the heuristic approach suggests a viable pathway for reducing complexity in future iterations of the project.

Proposed heuristic rules for wagon assignment

Assignment of wagons from the origin stations of requests based on specific assignment conditions and, in stations with the same origin request, selecting the request under selection priority criteria

Symbols	Priority criterion
$RU_{1,1}$	Lower inventory of freight
$RU_{1,2}$	Higher inventory of freight
$RU_{1,3}$	Higher commitment completion
$RU_{1,4}$	Lower commitment completion
$RU_{1,5}$	No assignment in this period

Proposed heuristic rules for wagon reassignment

Reassignment of wagons from each station to the origin of a request, while the station was the destination of that request or is within a specified distance from its origin, and ensuring that the type of wagons being reassigned is suitable for the freight of that request. When multiple requests satisfy these conditions, the request is chosen according to the following priority criteria:

Symbols	Priority criterion
$RU_{2,6}$	Higher inventory of freight at origin.
$RU_{2,7}$	Lower inventory of freight at origin
$RU_{2,8}$	Higher commitment completion
$RU_{2,9}$	Lower commitment completion
$RU_{2,10}$	Lower number of available wagons at origin
$RU_{2,11}$	Higher number of available wagons at its origin is prioritized.
$RU_{2,12}$	No reassignment in this period.

Figure 9: Heuristic rules for wagon assignment or re-assignment. Taken from Habibi et al. (2026) [14].

Reward Mechanism

The reward function is formulated to align the agent's behavior with corporate profitability. It is composed of four distinct weighted components:

1. Positive reinforcement for revenue generated from fulfilled freight deliveries.
2. Penalties associated with delivery delays (lateness).
3. Operational costs associated with the transportation of fully loaded wagons.
4. Variable costs incurred by the movement of empty wagons (positioning costs).

Neural Network Architecture

The actor-critic framework employs three fully connected hidden layers, each consisting of 256 neurons. Rectified Linear Unit (ReLU) functions are utilized for non-linear activation. The optimization process is conducted via the Adam optimizer, utilizing a

learning rate of 0.002 and a discount factor (γ) of 0.9. These parameters served as the baseline configuration for my model before empirical fine-tuning was performed to adapt to the specific requirements of locomotive selection.

Technical Contribution and Thesis Relevance

The work by Habibi et al. served as a foundational benchmark for this research. While my model focuses on a multi-locomotive selection environment built with the Gymnasium framework, the architectural parallels in the PPO implementation provided a robust starting point for determining network size and hyperparameters.

Critically, however, the Habibi model assumes a deterministic environment. A significant gap identified during this review is the absence of modeling for stochastic failures or external uncertainty—events that are ubiquitous in real-world railway operations. While my model improves upon the user-interface and visual action-tracking, the next phase of this thesis aims to integrate the heuristic-based dimensionality reduction seen in Habibi's work while introducing environmental uncertainty to better simulate the "real-world" drift between planned schedules and execution.

7 An actor-critic deep reinforcement learning approach for metro train scheduling with rolling stock circulation under stochastic demand

The study by Ying et al. [15] investigates an actor-critic Deep Reinforcement Learning (DRL) framework for metro train scheduling, with a specific focus on the rotation of a constrained fleet of rolling stock units. The system is modeled as a closed-loop network of stations $\mathbb{I} = \{1, 2, \ldots, I\}$, where I_0 represents the depot for train dispatching. The objective is to assign a fixed fleet of K trains to N service runs over a predefined horizon, ensuring efficient circulation under capacity constraints.

State Space Representation

The state tuple s_n at each stage n encapsulates the observed environment for the agent:

$$s_n = (\tau_{n,i}, \sigma_{n,i}, l_{n,i}) \tag{34}$$

where $\tau_{n,i}$ and $\sigma_{n,i}$ denote the arrival and departure times of service n at station i, respectively. The term $l_{n,i}$ represents the passenger accumulation remaining at station i following the departure of service n. While structurally similar to the work of Zhao et al. [11], this formulation places greater emphasis on the specific requirements of rolling stock planning.

Action Space Representation

The action vector x_n determined at stage n for the subsequent service $n+1$ is defined as:

$$x_n = (h_{n+1}, u_{n+1,i}, w_{n+1,i}) \tag{35}$$

where h_{n+1} is the headway between service n and $n+1$ at the dispatching point, $u_{n+1,i}$ is the section running time between station pairs $(i, i+1)$, and $w_{n+1,i}$ denotes the station dwell times. These parameters are subject to bound-clipping to maintain operational

feasibility. A critical assumption in this model is that the fleet size K is smaller than the total service requirements, thereby necessitating an effective circulation and reuse strategy.

Reward Mechanism

The reward function is a weighted multi-objective scalar derived from two perspectives:

1. Passenger Utility: A cost function $c_n^W = \zeta^W \sum_{i=1}^{I} c_{n,i}^W$ quantifies the waiting time for passengers. The factor ζ^W serves to monetize total waiting time based on established economic standards [16, 17].

2. Operator Efficiency: The operational cost c_n^V accounts for energy consumption and maintenance-related expenditures:

$$c_n^V = \sum_{i=1}^{I} \left(\zeta^F F(v_{n,i}) d_{i,i+1} + \zeta^T u_{n,i} + \zeta^D d_{i,i+1} \right) \tag{36}$$

   where $F(v_{n,i})$ represents energy consumption as a function of speed, and $\zeta^F, \zeta^T, \zeta^D$ are unit cost factors for fuel, time-related expenditures, and distance-related wear, respectively.

Neural Network Architecture

To mitigate the curse of dimensionality, the authors implement the Deep Deterministic Policy Gradient (DDPG) algorithm. The architecture employs two-layer artificial neural networks for both the actor and the critic, utilizing non-linear activation functions and the Adam optimizer [18]. A linear mapping function f is used to scale the bounded action space $[-1, 1]$ to real-world operational values.

Technical Contribution and Thesis Relevance

This paper serves as a primary reference for the implementation of action-value clipping and linear mapping within this thesis. Furthermore, the inclusion of multi-faceted maintenance costs in the reward function provided a methodological foundation for our model, despite our simplified parameter set.

Most significantly, Ying et al. highlight the inherent difficulty of achieving convergence in environments characterized by stochasticity and random failure events. This aligns with the findings of the present research: while the system demonstrates high operational performance, the training curves exhibit the characteristic volatility (as seen in Figure 10) common in high-uncertainty DRL applications. This research confirms that while reducing stochastic variables leads to smoother convergence, it results in a less robust model capable only of finding trivial solutions.

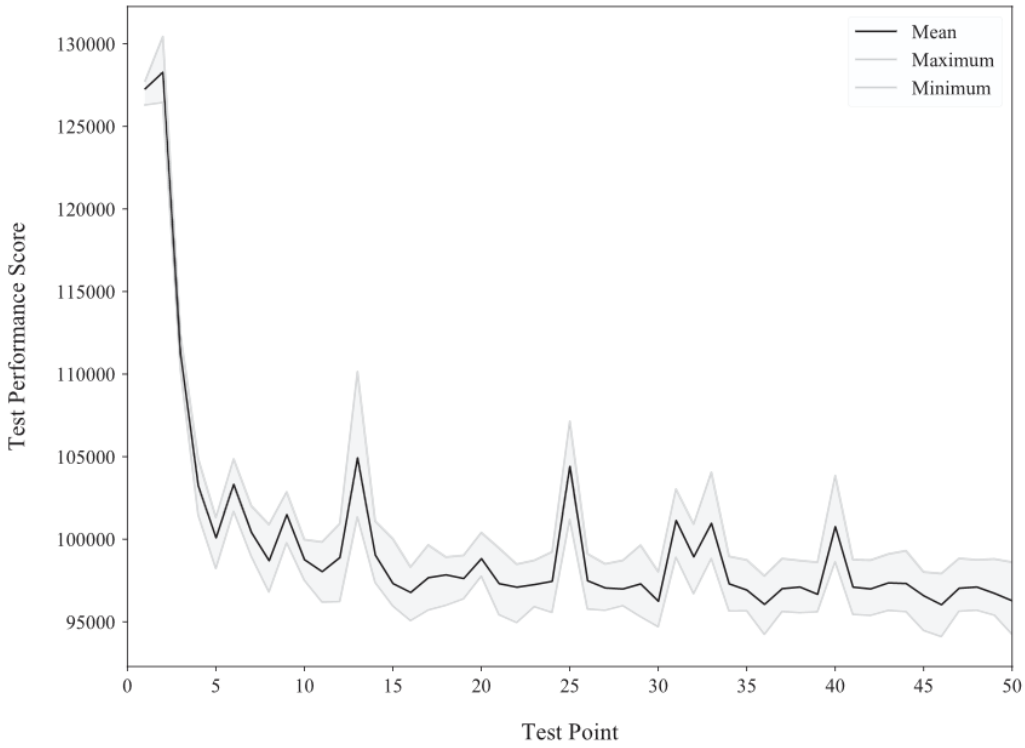


Figure 10: Test of the actor-critic agent during the training process.Taken from (Ying et al., 2020 [15])

8 Dynamic wagon assignment and reassignment for pick-up and delivery requests: A deep reinforcement learning framework to address uncertainty in locomotive availability

The research conducted by Habibi et al. (2026) [14] addresses the complexities of railway logistics through the application of Deep Reinforcement Learning (DRL), specifically utilizing the Proximal Policy Optimization (PPO) algorithm. The primary objective of their framework is the maximization of operational efficiency by ensuring the fulfillment of delivery requests while simultaneously minimizing logistical costs and temporal delays.

State Space Representation

The agent observes a discrete environment where the state s_t at any given time t is defined as:

$$s_t = (Bv_{t,m}^K, ya_{t,w,i})$$

This representation captures two critical components:

- The current inventory levels of freight across all pending requests.

- The distribution and availability of specific wagon types across the network's stations.

This state space design mirrors the requirements of complex railway environments, providing the necessary visibility into resource allocation and demand fulfillment.

Action Space Representation

A recurring challenge in railway scheduling—as observed in the model developed for this thesis—is the lack of scalability when the number of stations or assets increases. To mitigate the "curse of dimensionality" inherent in large-scale discrete action spaces, Habibi et al. introduce a set of twelve heuristic rules 11. These rules act as an abstraction layer for the agent, constraining the action space to high-level wagon assignment and re-assignment decisions. This approach contrasts with the raw weight-based selection mechanism utilized in my model, where the agent directly outputs weights for locomotive selection. While my model currently grants the agent greater freedom, the heuristic approach suggests a viable pathway for reducing complexity in future iterations of the project.

Proposed heuristic rules for wagon assignment

Assignment of wagons from the origin stations of requests based on specific assignment conditions and, in stations with the same origin request, selecting the request under selection priority criteria

Symbols	Priority criterion
$RU_{1,1}$	Lower inventory of freight
$RU_{1,2}$	Higher inventory of freight
$RU_{1,3}$	Higher commitment completion
$RU_{1,4}$	Lower commitment completion
$RU_{1,5}$	No assignment in this period

Proposed heuristic rules for wagon reassignment

Reassignment of wagons from each station to the origin of a request, while the station was the destination of that request or is within a specified distance from its origin, and ensuring that the type of wagons being reassigned is suitable for the freight of that request. When multiple requests satisfy these conditions, the request is chosen according to the following priority criteria:

Symbols	Priority criterion
$RU_{2,6}$	Higher inventory of freight at origin.
$RU_{2,7}$	Lower inventory of freight at origin
$RU_{2,8}$	Higher commitment completion
$RU_{2,9}$	Lower commitment completion
$RU_{2,10}$	Lower number of available wagons at origin
$RU_{2,11}$	Higher number of available wagons at its origin is prioritized.
$RU_{2,12}$	No reassignment in this period.

Figure 11: Heuristic rules for wagon assignment or re-assignment. Taken from Habibi et al. (2026) [14].

Reward Mechanism

The reward function is formulated to align the agent's behavior with corporate profitability. It is composed of four distinct weighted components:

1. Positive reinforcement for revenue generated from fulfilled freight deliveries.
2. Penalties associated with delivery delays (lateness).
3. Operational costs associated with the transportation of fully loaded wagons.
4. Variable costs incurred by the movement of empty wagons (positioning costs).

Neural Network Architecture

The actor-critic framework employs three fully connected hidden layers, each consisting of 256 neurons. Rectified Linear Unit (ReLU) functions are utilized for non-linear activation. The optimization process is conducted via the Adam optimizer, utilizing a

learning rate of 0.002 and a discount factor (γ) of 0.9. These parameters served as the baseline configuration for my model before empirical fine-tuning was performed to adapt to the specific requirements of locomotive selection.

Technical Contribution and Thesis Relevance

The work by Habibi et al. served as a foundational benchmark for this research. While my model focuses on a multi-locomotive selection environment built with the Gymnasium framework, the architectural parallels in the PPO implementation provided a robust starting point for determining network size and hyperparameters.

Critically, however, the Habibi model assumes a deterministic environment. A significant gap identified during this review is the absence of modeling for stochastic failures or external uncertainty—events that are ubiquitous in real-world railway operations. While my model improves upon the user-interface and visual action-tracking, the next phase of this thesis aims to integrate the heuristic-based dimensionality reduction seen in Habibi's work while introducing environmental uncertainty to better simulate the "real-world" drift between planned schedules and execution.

References

[1] Gely P. Basharin, Amy N. Langville, and Valeriy A. Naumov. The life and work of A. A. Markov. *Linear Algebra and its Applications*, 386:3–26, 2004.

[2] Hao Jiang, Renjie Gui, Zhen Chen, Liang Wu, Jian Dang, and Jie Zhou. An improved sarsa(λ) reinforcement learning algorithm for wireless communication systems. *IEEE Access*, PP:1–1, 08 2019.

[3] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, Cambridge, Massachusetts, second edition, 2018.

[4] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin A. Riedmiller. Playing atari with deep reinforcement learning. *CoRR*, abs/1312.5602, 2013.

[5] John Schulman, Sergey Levine, Pieter Abbeel, Michael I. Jordan, and Philipp Moritz. Trust region policy optimization. In Francis R. Bach and David M. Blei, editors, *Proceedings of the 32nd International Conference on Machine Learning, ICML 2015, Lille, France, 6-11 July 2015*, volume 37 of *JMLR Workshop and Conference Proceedings*, pages 1889–1897. JMLR.org, 2015.

[6] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, Cambridge, UK, 2004.

[7] Jonathan Richard Shewchuk. An introduction to the conjugate gradient method without the agonizing pain. Technical report, Carnegie Mellon University, Pittsburgh, PA, USA, 1994.

[8] Joshua Achiam. Spinning up in deep reinforcement learning. `https://github.com/openai/spinningup`, 2018.

[9] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *CoRR*, abs/1707.06347, 2017.

[10] Yuhua Yang, Haoyang Huo, Nikola Bešinović, Yichen Sun, and Shaoquan Ni. An advanced learning environment and a scalable deep reinforcement learning approach for rolling stock circulation on urban rail transit line. *Transportation Research Part C: Emerging Technologies*, 171:104976, 2025.

[11] Xiaoli Zhao, Dewei Li, and Xinyu Bao. A deep reinforcement learning approach for integrated optimization of train scheduling and rolling stock circulation planning. *Computers & Industrial Engineering*, 213:111784, 2026.

[12] Xiangyu Su, Yixiang Yue, Bin Guo, Zanyang Cui, and Boyan Li. Solving high-speed rolling stock planning with an improved lstm-ppo reinforcement learning algorithm. *Transportmetrica A Transport Science*, 2026.

[13] Manuel Bröchin, Reto Ramseier, and Kaspar Schüpbach. Rolling stock planning with maintenance constraints by a rolling horizon approach. 04 2023.

[14] Parivash Habibi, Mohammad Tamannaei, and Hamid Zarei. Dynamic wagon assignment and reassignment for pick-up and delivery requests: A deep reinforcement learning framework to address uncertainty in locomotive availability. *Applied Soft Computing*, 187:114351, 2026.

[15] Cheng shuo Ying, Andy H.F. Chow, and Kwai-Sang Chin. An actor-critic deep reinforcement learning approach for metro train scheduling with rolling stock circulation under stochastic demand. *Transportation Research Part B: Methodological*, 140:210–235, 2020.

[16] Department for Transport (DfT). Tag unit 3.5.6 values of time and vehicle operating costs, transport analysis guidance data book. Technical report, Department for Transport, United Kingdom, 2014.

[17] A. H. F. Chow and A. Pavlides. Cost functions and multi-objective timetabling of mixed train services. *Transportation Research Part A*, 113:335–356, 2018.

[18] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, Cambridge, MA, 2016.